

Развёртывание системы BHS.CRG

Система исполнительной документации BHS.CRG

Версия документа: 2026-09-23

Инструкция по развёртыванию BHS.CRG

Система генерации исполнительной документации для электромонтажных проектов. Развёртывание — через **Docker Compose** на сервере **Linux**: весь стек (веб-интерфейс, сервер приложений, база данных, объектное хранилище, модель распознавания) поднимается несколькими командами.

Этот документ — для администратора/инженера, разворачивающего систему. Для работы с системой см. «Инструкцию пользователя» и «Инструкцию администратора».

1. Архитектура развёртывания

Компонент	Образ / технология	Назначение
web	nginx + собранный React-SPA	Веб-интерфейс; проксирует /api на сервер
api	ASP.NET Core 10 + Typst	Сервер приложений, REST API, генерация PDF
postgres	PostgreSQL 18	Основная база данных
garage	Garage	Объектное хранилище (сканы, наборы данных, готовые файлы)
ollama	Ollama	Локальная модель распознавания сканов. По умолчанию не разворачивается — включается профилем, см. §7

Вспомогательные одноразовые сервисы: `garage-init` (готовит хранилище: раскладка, ключ, bucket, права), `ollama-init` (загружает модель распознавания — вместе с `ollama`, только при включённом профиле).

До версии 0.160.0 хранилищем был MinIO. Он архивирован разработчиком (community-версия больше не собирается и не получает исправлений безопасности), поэтому система переведена на Garage. Переход делает `update.sh` сам, перенося файлы, — см. §8.7.

Схематично:



Генерация: PDF собирается движком **Typst** (входит в образ `api`). Формат **DOCX в текущей версии не поддерживается** — только PDF.

2. Требования

- **HTTPS обязателен.** Система работает с паролями и токенами доступа; по открытому HTTP они идут по сети в читаемом виде, и перехват одного запроса даёт полный доступ к системе от имени пользователя. Стек TLS не терминирует сам — перед сервисом `web` ставится обратный прокси с сертификатом (nginx, Caddy, Traefik) либо терминатор периметра. Порядок с нуля — **Приложение В**. Без него допустима только установка, целиком закрытая внутри доверенной сети, и даже там часть интерфейса работать не будет: браузер отключает вне защищённого контекста буфер обмена, снимок экрана в форме сообщения об ошибке и `crypto.randomUUID` (B.6). См. также §11.
- **ОС:** Linux (Ubuntu 22.04+/Debian 12+ и совместимые).
- **Docker Engine 24+** и **Docker Compose v2 или новее** — тот, что вызывается как `docker compose` (через пробел), а не `docker-compose`. Номер у плагина давно ушёл вперёд движка: свежая установка отвечает `v5.x`, и это не ошибка — важно, что не `v1`. На чистом сервере их ещё нет: установка из официального репозитория — **Приложение А**. Там же сказано, чего не ставить (`docker.io`, `docker-compose v1`, `snap`) и почему выбор не того пакета выясняется не сразу.
- **Ресурсы:**
 - Базовый стек (без Ollama): от **2 CPU / 4 ГБ RAM / 10 ГБ диска**. Столько и требует установка по умолчанию — локальное распознавание не разворачивается, пока его не включат.
 - С Ollama и моделью `qwen2.5vl:7b` (включается по §7): дополнительно **~8 ГБ RAM** и **~6 ГБ диска** под модель; желательна видеокарта, но работает и на CPU (медленнее).
- Доступ в интернет на этапе сборки (загрузка образов, `Typst`, `npm`/`NuGet`-пакетов) и, при использовании веб-поиска документов качества, в рантайме.

Проверка окружения:

```
docker --version
docker compose version
```

Обе команды должны ответить номером версии. Ответ `'compose' is not a docker command` означает, что плагин Compose не установлен: см. **Приложение А**.

3. Быстрый старт

Исходный код для установки **не нужен**: система поставляется готовыми образами из GHCR.

Скриптом (рекомендуется)

```
mkdir bhs-crg && cd bhs-crg
curl -LO https://github.com/pavru/bhs.crg/releases/latest/download/install.sh
less install.sh          # посмотрите, что он делает – привычка полезная
bash install.sh
```

Скрипт задаст **три вопроса** (публичный адрес, порт, нужно ли локальное распознавание), спросит почту и пароль администратора (не короче 8 знаков, цифра, строчная и заглавная **латинская** буква — так требует сервер, кириллический пароль он не примет) — и дальше сделает всё сам: скачает файлы выпуска, создаст `.env` со случайными паролями и ключами нужного формата, подготовит каталог копий, поднимет систему, дождётся её готовности и **сразу заведёт администратора**.

Последнее — не удобство, а закрытая дверь: пока первой учётной записи нет, страница регистрации доступна любому, кто дотянется до адреса (§5), и администратором станет тот, кто успел первым.

`curl ... | bash` мы намеренно не предлагаем: исполнять скачанное вслепую на сервере, где будет лежать документация целого предприятия, — плохая привычка. Скачайте, посмотрите, запустите.

Полезные ключи:

Ключ	Зачем
<code>bash install.sh 0.160.0</code>	поставить конкретный выпуск, а не последний
<code>--dir /opt/bhs-crg</code>	поставить в другой каталог
<code>--dry-run</code>	показать план и выйти, ничего не меняя
<code>--non-interactive</code>	без вопросов; ответы из переменных окружения (<code>ADMIN_EMAIL</code> , <code>ADMIN_PASSWORD</code> , <code>WEB_PORT</code> , <code>APP_PUBLIC_URL</code> , <code>WITH_OLLAMA</code>) — для Ansible и подобного
<code>--reverse-proxy docs.example.ru</code>	подготовить конфиг nginx для этого домена — см. ниже

Повторный запуск в каталоге, где уже есть `.env` , **отказывается работать**: там стоит система, и перезапись затёрла бы её ключи вместе с доступом к данным. Для обновления есть `./update.sh` .

Ключ `--reverse-proxy`

`bash install.sh --reverse-proxy docs.example.ru` дополнительно:

- кладёт рядом с установкой готовый `reverse-proxy.conf` — образец из Приложения В с подставленными доменным именем, портом и пределом размера тела (он берётся из `BACKUP_MAX_ARCHIVE_MB` плюс запас);
- ставит `WEB_BIND=127.0.0.1` , закрывая порт снаружи: за прокси публиковать его не нужно, иначе к системе можно подключиться в обход HTTPS;
- ставит `APP_PUBLIC_URL=https://docs.example.ru` , если адрес не задан явно;
- печатает команды, которыми конфиг ставится.

Сам nginx скрипт не трогает — ни копирования в `/etc/nginx` , ни ссылки в `sites-enabled` , ни удаления `default` , ни перезагрузки, ни `certbot` . Это не осторожность ради осторожности: на хосте могут работать чужие сайты, и автоматическая перезагрузка чужого nginx способна их уронить. Прав `root` ключу поэтому не нужно вовсе.

Руками

Если хочется видеть каждый шаг — порядок тот же, что делает скрипт:

```
# 1. Взять со страницы выпуска (https://github.com/pavru/bhs.crg/releases/latest)
#   docker-compose.yml, garage.toml и .env.example — они приложены к релизу.
mkdir bhs-crg && cd bhs-crg
curl -LO https://github.com/pavru/bhs.crg/releases/latest/download/docker-compose.yml
curl -LO https://github.com/pavru/bhs.crg/releases/latest/download/garage.toml
curl -Lo .env.example https://github.com/pavru/bhs.crg/releases/latest/download/env.example

# 2. Подготовить конфигурацию
cp .env.example .env
nano .env                # задать пароли, ключи хранилища, JWT-секрет и APP_VERSION (см.
раздел 4)

# 3. Каталог резервных копий — владельцем должен быть пользователь app из образа
mkdir -p backups && sudo chown 1654:1654 backups

# 4. Запустить весь стек (образы скачаются из GHCR)
docker compose up -d

# 5. Проверить статус и СРАЗУ завести администратора (§5) — пока этого не сделано,
#   страница регистрации открыта всем
docker compose ps
docker compose logs -f api
```

garage.toml — не «до кучи»: compose монтирует его рядом с собой. Файла нет — Docker создаёт на его месте каталог, и хранилище падает с `IO error: Is a directory`, то есть сообщением, по которому причину не найти.

Дальше в инструкции команды пишутся в виде `docker compose -f deploy/docker-compose.yml ...` — это форма для запуска из клона репозитория. Если вы поставили систему из релиза, файл лежит рядом, и `-f deploy/docker-compose.yml` можно опускать.

Сборка из исходников (разработка, проверка правки до релиза, установка без готовых образов приложения):

```
git clone https://github.com/pavru/bhs.crg.git && cd bhs.crg
cp deploy/.env.example deploy/.env && nano deploy/.env
docker compose -f deploy/docker-compose.yml -f deploy/docker-compose.build.yml up -d --build
```

Образы публичные — `docker login ghcr.io` для установки не нужен: пакеты наследуют видимость репозитория, и проверено это на первом же выпуске (анонимный pull обоих образов работает). Если однажды pull всё же потребует авторизации, значит видимость пакета изменили вручную — вернуть её можно в *Packages* → *Package settings* → *Change visibility*.

Из нашего реестра приходят три образа, а не два: `api`, `web` и `garage-init` — одноразовый сервис, который готовит хранилище при старте. Само хранилище (`garage`) и база (`postgres`) тянутся с Docker Hub. Доступ к `ghcr.io` нужен **и** при сборке из исходников: оверлей собирает приложение, но образы хранилища берутся готовыми.

В нашем реестре остаётся и копия **MinIO** — прежнего хранилища (`ghcr.io/pavru/minio`, `ghcr.io/pavru/mc`). Она нужна установкам, которые придут на обновление со старых версий: именно ею `update.sh` переносит файлы (§8.7). Удалять её нельзя, даже когда все перейдут.

После запуска веб-интерфейс доступен по адресу **`http://СЕРВЕР:8080`** (порт задаётся `WEB_PORT` в `.env`).

При первом старте сервер `api` автоматически применяет миграции базы данных и создаёт роли `Admin / User`. Отдельных команд миграции выполнять не нужно.

4. Конфигурация (`.env`)

Скопируйте `.env.example` → `.env` и заполните. Файл `.env` содержит секреты и **не** хранится в репозитории. При установке из релиза оба файла лежат в рабочем каталоге; в клоне репозитория — в `deploy/`.

Переменная	Назначение	Обязательно
APP_VERSION	Версия системы: под этим тегом тянутся образы <code>api</code> и <code>web</code> из GHCR. Без неё стек не поднимется — умолчания намеренно нет, иначе установка молча работала бы на непонятной версии. Номер берётся со страницы выпусков ; в шаблоне из релиза он уже проставлен	да
POSTGRES_DB / POSTGRES_USER / POSTGRES_PASSWORD	Параметры БД. Пароль из примера останавливает запуск	да (сменить пароль)
GARAGE_KEY_ID / GARAGE_SECRET	Ключ доступа к хранилищу — он открывает все файлы системы . Формат жёсткий: <code>GK</code> и 24 шестнадцатеричных знака у идентификатора, ровно 64 у секрета. Незаданные останавливают запуск	да (при обновлении создаются скриптом)
GARAGE_BUCKET	Имя bucket для файлов. Пустое значение останавливает запуск	да (по умолч. <code>bhs-crg</code>)
GARAGE_RPC_SECRET / GARAGE_ADMIN_TOKEN	Внутренние секреты самого хранилища; сервер приложений их не читает. RPC-секрет — ровно 64 шестнадцатеричных знака, иначе хранилище не стартует (сообщение будет в <code>docker compose logs garage</code> , не в <code>api</code>)	да (при обновлении создаются скриптом)
JWT_KEY	Секрет подписи токенов, ≥ 32 символов	да
MODULES_ENABLED	Какие предметные модули включены на экземпляре, через запятую. От набора зависят разделы, права и состав данных, поэтому задаёт его поставка, а не администратор. Пустое значение — умолчание <code>id</code> (исполнительная документация), а не «все»: пустая строка в <code>.env</code> чаще всего след невычищенной правки. Код, которого нет в сборке, останавливает запуск — чтобы опечатка не обернулась пропавшим разделом	нет (по умолч. <code>id</code>)
WEB_PORT	Порт веб-интерфейса на хосте	нет (по умолч. <code>8080</code>)
WEB_BIND	Адрес публикации веб-интерфейса. Поставили перед системой прокси с TLS (Приложение В) — ставьте <code>127.0.0.1</code> : порт перестает отвечать снаружи, оставаясь доступным прокси	нет (по умолч. <code>0.0.0.0</code>)
FORWARDED_TRUSTED_NETWORKS	Сети, чьему заголовку <code>X-Forwarded-For</code> сервер доверяет (CIDR через запятую). Пусто — петля и частные диапазоны. Негодная запись останавливает запуск	нет
APP_PUBLIC_URL	Публичный адрес системы (напр. <code>https://docs.example.ru</code>) — подставляется ссылкой в письма	да, если нужна почта
BACKUP_DIR	Каталог резервных копий на хосте : сюда система их кладёт и отсюда восстанавливает. Каталог должен	нет (по умолч. <code>./backups</code>)

Переменная	Назначение	Обязательно
	принадлежать пользователю <code>app</code> (uid 1654) — иначе запуск останавливается с указанием нужной команды	
<code>BACKUP_KEEP_COUNT</code>	Сколько копий держать в каталоге (1...1000). Предел достигнут — новая копия не создаётся (отказ с объяснением), старые не удаляются сами. Негодное значение останавливает запуск	нет (по умолч. <code>10</code>)
<code>BACKUP_MAX_ARCHIVE_MB</code>	Предел размера копии, загружаемой через браузер , МБ (16...10240). Ей же задаётся <code>client_max_body_size</code> веб-сервера — второе значение править не нужно. Копия крупнее переносится файлом в <code>BACKUP_DIR</code> . Негодное останавливает запуск	нет (по умолч. <code>500</code>)
<code>COMPOSE_PROFILES</code>	Какие необязательные сервисы поднимать; сегодня значение одно — <code>ollama</code> . Пусто — локальное распознавание не разворачивается	нет (пусто)
<code>OLLAMA_MODEL</code>	Модель локального распознавания. Пусто — на новой установке Ollama выключена и в самом приложении: движок участвует в работе ровно тогда, когда у него выбрана модель. Задаётся ВМЕСТЕ с <code>COMPOSE_PROFILES=ollama</code> . Оговорка для существующих установок: если настройки интеграций хоть раз сохраняли в интерфейсе, модель записана в базу, а база сильнее переменной — выключать движок тогда надо там же	нет (пусто)
<code>ANTHROPIC_API_KEY</code>	Распознавание реквизитов (Claude)	опц.
<code>GEMINI_API_KEY</code>	Распознавание реквизитов (Gemini)	опц.
<code>SERPER_API_KEY</code>	Веб-поиск документов качества	опц.

Сгенерировать надёжный `JWT_KEY` :

```
openssl rand -base64 48
```

Важно: обязательно смените пароли БД/MinIO и `JWT_KEY` перед вводом в эксплуатацию. Значения из примера — только для ознакомления.

`JWT_KEY` проверяется при запуске: **приложение не поднимется**, если значение не задано, короче 32 байт или оставлено из файла-примера. Это сделано намеренно — иначе установку легко ввести в эксплуатацию, не заметив, что ключ подписи не менялся.

Так же проверяются настройки хранилища и параметры БД. Раньше при незадаанных значениях система поднималась молча, а отказ приходил позже и чужими словами: ошибка клиента MinIO при первой загрузке файла, `Host can't be null` от драйвера БД. Теперь запуск останавливается с указанием, что именно задать, если не заполнено любое из:

- `GARAGE_KEY_ID` , `GARAGE_SECRET` , `GARAGE_BUCKET` (их проверяет сервер при старте);
- `GARAGE_RPC_SECRET` — его проверяет само хранилище, отказом стартовать;

- любая часть строки подключения — `POSTGRES_DB` , `POSTGRES_USER` , `POSTGRES_PASSWORD` или адрес сервера. Строка разбирается, а не проверяется на пустоту целиком: незаданная переменная даёт не пустую строку, а строку с пустым полем (`...;Password=`), и проверка «не пусто» такое пропустила бы — то есть ровно в том случае, с которым и сталкиваются, её бы не было.

Сюда же — `BACKUP_KEEP_COUNT` и `BACKUP_MAX_ARCHIVE_MB` : не число или значение вне допустимого диапазона останавливает запуск. Понять «500 МБ» как 500 и промолчать было бы хуже, чем отказать: описка в `.env` тихо вернула бы умолчание на установке, где предел специально поднимали. Контейнер `web` на негодном значении тоже не поднимается.

Значение, оставленное из файла-примера, тоже останавливает запуск. Смотреть `docker compose logs api` .

Пароль БД обязателен: вход без пароля (`trust` , `.pgpass`) наша поставка не разворачивает, а СУБД, доступная по сети без пароля, — сама по себе дефект.

Если система уже работала на значении из примера, после смены ключа **отзовите refresh-токены**: выданные ранее перестанут быть доверенными, пользователи войдут заново. Это ожидаемое поведение.

Каталог резервных копий (`BACKUP_DIR`)

Копии лежат **на хосте**, в каталоге, примонтированном в контейнер `api` . Это точка монтирования, а не том Docker, и выбрано так намеренно: по этому пути копию увозят `rsync` -ом, приносят с другого сервера и кладут рядом руками — переезд системы делается именно этим.

Каталог создаётся один раз, до первого запуска. Контейнер работает **не от root**, поэтому владелец имеет значение:

```
mkdir -p backups
sudo chown 1654:1654 backups      # uid пользователя app в образе
```

Выполнять — рядом с `docker-compose.yml` , и это не придирка: относительный путь Compose считает не от вашего текущего каталога, а от каталога проекта — того, где лежит `compose`-файл. В установке из релиза это одно и то же; в клоне репозитория команды даются из корня (`-f deploy/docker-compose.yml`), а каталог система будет искать в `deploy/` . Создав `backups` в корне, вы получите ровно тот отказ, ради предотвращения которого этот раздел и написан: Docker молча заведёт `deploy/backups` от `root` , и сервер не поднимется. Либо перейдите в `deploy` , либо задайте `BACKUP_DIR` абсолютным путём — например `/var/lib/bhs-crg/backups` .

Пропустить этот шаг можно, но недолго: недостающий каталог Docker создаст сам — от `root` , — и сервер откажется стартовать, назвав и путь, и команду. Отказ на старте здесь выбран вместо отказа при первой копии: узнать о неверных правах в день, когда копия понадобилась, — худший из возможных вариантов.

Что в каталоге лежит:

- копии, снятые из интерфейса, — имя вида `crg-backup-20260822-101500-v0.141.0.zip` (дата, время и версия системы, которой копия снята);

- копии, принесённые с другого сервера. Достаточно положить файл в каталог: он появится в списке **Настройка системы** → **Настройки** → **Резервное копирование** и восстанавливается оттуда же.

Число копий ограничено `BACKUP_KEEP_COUNT` (по умолчанию 10). Предел достигнут — новая копия не создаётся, и система говорит об этом: старые она не удаляет. Автоудаление однажды унесло бы ту единственную копию, которая понадобится; уборка по расписанию появится вместе с плановыми копиями.

Почтовые сценарии (сброс пароля, подтверждение email)

Письма со **ссылками** — самостоятельный сброс пароля, подтверждение адреса, смена email, отправка крупного комплекта — требуют **двух** настроек:

1. `APP_PUBLIC_URL` в `deploy/.env` — внешний адрес, по которому пользователи открывают систему (именно он подставляется в ссылку письма). Без него письма со ссылками **не отправляются**, а действие возвращает понятную ошибку (пользователь бесполезного письма не получит).
2. **SMTP** — параметры почтового сервера задаёт администратор в UI: **Настройка системы** → **Настройки** → **Почта** (host/port/логин/шифрование). Без SMTP письма не уходят.

Ключи шифрования одноразовых токенов (сброс/подтверждение) хранятся на томе `dp_keys` (в `compose` уже настроен) — поэтому ссылки переживают перезапуск контейнера `api`. Срок жизни access-токена — 60 мин (обновляется автоматически); настраивается `Jwt__AccessTokenMinutes` при необходимости.

5. Первый вход (создание администратора)

1. Откройте **`http://СЕРВЕР:8080`**. Пока в системе нет ни одного пользователя, вместо формы входа открывается экран **«Первый вход»**: email, имя, пароль и его подтверждение.
2. Заполните и нажмите **«Создать администратора и войти»**. Учётная запись получит роль **Администратор**, и вход выполнится сразу — второй раз вводить те же данные не нужно.
3. После этого **публичная регистрация закрывается сама**: по тому же адресу снова обычная форма входа. Остальные учётные записи заводит администратор — **Настройка системы** → **Пользователи** (см. «Инструкцию администратора»).

Это окно открыто всем. От `ip -d` и до создания первой учётной записи экран «Первый вход» доступен любому, кто дотянется до адреса, и администратором станет тот, кто успел первым. Делайте первый вход сразу после запуска — а если система смотрит наружу, то **до** того, как открыть к ней доступ.

Если к интерфейсу доступа нет, а к хосту есть (ставите по SSH, порт наружу ещё закрыт), администратора можно завести запросом — это тот же путь, которым идёт форма:

```
curl -i -X POST http://localhost:8080/api/auth/register -H "Content-Type: application/json"
-d '{"email":"admin@example.ru","password":"ВашПароль1","displayName":"Администратор"}'
```

Пароль — не короче 8 символов, с цифрой, строчной и заглавной буквой. `-i` здесь обязателен: при успехе тело ответа **пустое**, и без кода состояния удача неотличима от молчаливого отказа. `200 OK` — готово; `400` — в теле сказано, чем плох пароль; `403` — учётная запись уже есть.

6. Порты

Сервис	Внутренний порт	Порт на хосте	Примечание
web (nginx)	8080	<code>WEB_PORT</code> (8080)	Основной вход в систему
api	8080	—	Доступен только внутри сети Compose
postgres	5432	—	Наружу не публикуется
garage (S3)	3900	—	Наружу не публикуется
garage (admin)	3903	—	Наружу не публикуется
ollama	11434	—	Наружу не публикуется

Наружу выведен только веб-интерфейс. Базу, хранилище и Ollama публиковать не следует.

Веб-консоли у хранилища нет. У MinIO она была и публиковалась на петлю; Garage её не имеет вовсе, и наружу не выводится ничего. Посмотреть состояние хранилища или список bucket'ов можно командой внутри контейнера:

```
docker compose exec garage /garage status
docker compose exec garage /garage bucket list
docker compose exec garage /garage bucket info bhs-crg
```

Потери доступа это не создаёт: всё, что делалось через консоль, делается этими командами, а файлы системы по-прежнему открываются через интерфейс приложения.

Внутренний порт `web` — 8080, а не 80: nginx в образе работает от непривилегированного пользователя. Порт на хосте (`WEB_PORT`) прежний, для обратного прокси ничего не меняется.

Заголовки безопасности

Сервис `web` отдаёт `Content-Security-Policy`, `X-Content-Type-Options`, `X-Frame-Options`, `Referrer-Policy`, `Permissions-Policy` и `Strict-Transport-Security` (см. `deploy/nginx.conf`). Политика содержимого разрешает скрипты **только с адреса самого приложения** — сторонние CDN и внедрённые в данные инлайновые скрипты не выполняются.

Если вы правите фронтенд под себя и подключаете стороннюю библиотеку по ссылке, она будет заблокирована — это ожидаемо. Правильный путь: положить библиотеку в сборку, а не ослаблять политику. HSTS браузер применяет только поверх HTTPS, поэтому заголовок безвреден до появления терминатора TLS и включается сам, когда тот появится.

7. Опциональные компоненты

Распознавание сканов (документы качества)

- **Ollama (локально):** по умолчанию **не разворачивается** (issue #824) — образ с моделью занимает ~6 ГБ диска и до 10 ГБ памяти, а установке с облачным распознаванием (или вовсе без документов качества) они не нужны. Включается двумя строками в `.env` и обычным `up -d`:

```
# в .env раскомментировать ОБЕ:
COMPOSE_PROFILES=ollama      # поднимать контейнеры ollama и ollama-init
OLLAMA_MODEL=qwen2.5vl:7b    # какую модель качать и какой пользоваться приложению

docker compose -f deploy/docker-compose.yml up -d
docker compose -f deploy/docker-compose.yml logs -f ollama-init # загрузка модели, ~6 ГБ
```

Если ваша сборка Compose не подхватывает `COMPOSE_PROFILES` из `.env` (ранние выпуски v2 читали эту переменную только из окружения), то же самое даёт флаг: `docker compose -f deploy/docker-compose.yml --profile ollama up -d`. Признаком именно этой беды один: `up -d` отработал без единой ошибки, а контейнеров `ollama` в `ps` нет.

Что включилось, видно в интерфейсе: **Настройка системы** → **Настройки** → **Поиск и распознавание** (Ollama перестаёт числиться ненастроенной) и в состоянии системы — появляется компонент «Ollama (распознавание)».

Загрузить или сменить модель вручную:

```
docker compose -f deploy/docker-compose.yml exec ollama ollama pull qwen2.5vl:7b
```

Выключить обратно: закомментировать обе строки в `.env` — и убрать контейнеры **явно**:

```
docker compose -f deploy/docker-compose.yml rm -sf ollama ollama-init
```

И если настройки интеграций когда-либо сохраняли в интерфейсе — снимите Ollama там же (**Настройка системы** → **Настройки** → **Поиск и распознавание**): сохранённая модель лежит в базе, а база сильнее переменной окружения. Иначе приложение продолжит считать движок включённым и мониторинг будет исправно сообщать, что он не отвечает.

Одной правки `.env` мало, и это проверено: контейнер отключённого профиля не останавливает ни `docker compose up -d` (даже с `--remove-orphans`), ни `docker compose down`. Последний уберёт остальные контейнеры, споткнётся о сеть (`Resource is still in use`) и оставит `ollama` работать; на стенде `down -v` снёс всё прочее вместе с томами, а она продолжила работать как ни в чём не бывало. Профиль решает, что поднимать, а не что гасить — молча остановленный им контейнер выглядел бы выключенным, а память и диск занимал бы дальше. Пока контейнер существует, он живёт своей жизнью: с политикой `restart: unless-stopped` он вернётся и после перезагрузки хоста. Том `ollama_data` со скачанной моделью тоже остаётся — освободить место отдельно: `docker volume rm bhs-crg-ollama_data` (имена — `docker volume ls`).

Строки именно две, и они про разное: первая решает, поднимать ли контейнеры, вторая — берёт ли Ollama в работу приложение. Раскомментировать одну — не поломка, но лишний круг: только профиль даёт контейнер вхолостую (скажет `docker compose logs ollama-init`), только модель — предупреждение в колокольчике «Ollama (распознавание): недоступна», потому что приложение ждёт движок, которого на хосте нет.

- **Облачные модели:** задайте `ANTHROPIC_API_KEY` и/или `GEMINI_API_KEY` в `.env`. Порядок перебора движков настраивается в коде (`Recognition:Order`) и в разделе настроек интеграций.

Веб-поиск документов качества

- Задайте `SERPER_API_KEY` (`serper.dev`) в `.env`. Без ключа поиск в интернете недоступен, остальная работа с документами качества — да.

Ключи интеграций можно также задавать/менять в самом приложении: **Настройка системы** → **Настройки** → **Поиск и распознавание**. В базе они хранятся **зашифрованными** (ключами Data Protection с тома `dp_keys`) и маскируются при чтении через API. Ключи, заданные переменными окружения, шифровать нечем и не нужно — они и так вне базы.

Следствие: при потере тома `dp_keys` расшифровать сохранённые ключи и пароль SMTP не получится — их надо будет ввести заново. Пока том просто недоступен (не примонтирован, переехал путь), сохранённые значения не портятся: приложение покажет их как незадаанные, но перезаписывать не станет — вернёт том, и они снова читаются.

Пароль SMTP привязан к серверу. Смена хоста, порта, пользователя или снятие шифрования обнуляют сохранённый пароль: его надо ввести заново. Это не придирка — иначе сохранение чужого адреса с пустым полем пароля отправило бы туда сохранённый пароль с первым же письмом. По той же причине проверка связи с другим сервером требует ввести пароль явно.

8. Обновление системы

Обновление выполняется **на хосте, из командной строки**. Кнопки «Обновить» в интерфейсе нет и не будет — это решение, а не недоделка: чтобы приложение пересобирало собственный стек, контейнеру нужен доступ к сокету Docker, а это превращает взлом веб-части во взлом всего сервера. Разбор с проверками — [issue #814](#).

8.1. Одной командой

```
# Скрипт лежит в каждом выпуске рядом с docker-compose.yml (в клоне репозитория –
deploy/update.sh)
VER=0.146.0
curl -fsSL https://github.com/pavru/bhs.crg/releases/download/v$VER/update.sh -o update.sh
chmod +x update.sh

./update.sh $VER
```

Запускать **из каталога установки** — там, где лежат ваши `docker-compose.yml` и `.env` (в установке из клона это `deploy/`).

Скрипт берёт на себя механику и **останавливается там, где нужно решение человека**. Это и есть его устройство: «как-нибудь слить» ваш `compose`-файл или «подставить что-нибудь» в новую переменную опаснее, чем инструкция, — инструкция хотя бы не притворяется, что справилась.

Что делает сам, в этом порядке:

1. Предполёт: есть `docker compose`, файлы на месте, установка видна командам, целевой выпуск существует и не ниже текущей версии.
2. Сверяет сам себя с `update.sh` целевого выпуска.
3. Смотрит в базу, не идут ли чужие фоновые задачи (перезапуск их оборвёт).
4. Забирает `docker-compose.yml` и `env.example` целевой версии в каталог `new/`.
5. Сверяет ваш `compose`-файл с файлом версии, а список переменных — с новым шаблоном: те из новых, значение которых и так подставилось бы, вписывает в `.env` сам, об остальных спрашивает. Печатает разовые оговорки — только те рубежи, которые ваше обновление пересекает.
6. Снимает дампы базы в `backups/pre-update-<с>-<на>-<дата>.dump` — на живой базе, **до** любой остановки контейнеров.
7. Проверяет разбор нового `compose` и **загружает образы** (до трёх попыток: обрыв связи с реестром — самый частый сбой, а повтор здесь ничего не стоит) — всё ещё на работающей старой версии.
8. Снимает снимок списка миграций (`migrations.prev-<дата>`), сохраняет прежние `docker-compose.yml` и `.env` с датой, переключает `APP_VERSION`, выполняет `up -d`.
9. Ждёт, пока `/api/version` ответит **целевой** версией (до 7 минут: `api` поднимается 20 с плюс 30 попыток `healthcheck` по 10 с), и печатает сводку: было → стало, где дампы, где прежние файлы.

Порядок именно такой, чтобы всё, что может отказать — недоступный реестр, неразобранный `compose`, недостающая переменная, — случилось **до** первой остановки контейнеров. Дампы снимаются после сверок намеренно: сверки — самые частые остановки, и снимать копию, чтобы через минуту упереться в незаполненную переменную, незачем; контейнеров к этому моменту всё равно никто не трогал. `APP_VERSION` в рабочий `.env` пишется только после успешной загрузки образов: оборвись сеть раньше, в файле стояла бы новая версия при старых образах — состояние, из которого не выйти ни одной командой.

Сколько займёт. На стенде обновление 0.137.0 → 0.140.0 заняло 19 секунд, из них система была недоступна около 15: сперва `API` отвечал `502` (веб-часть ещё старая, `api` уже пересоздаётся), потом на несколько секунд переставал открываться и сам адрес, пока пересоздавался `web`. Тяжёлая миграция на большой базе это время удлинит — насколько, заранее не знает никто.

8.2. Почему скрипт остановился

Каждая остановка — вопрос, на который может ответить только человек. Ниже — что она значит и что делать. Во всех случаях, кроме последнего, **система продолжает работать на прежней версии**.

«В системе есть незавершённые фоновые задачи». Перезапуск оборвёт чужую сборку комплекта или распознавание: сами они не возобновятся, новая версия при старте пометит их неудавшимися. Пилуля задач в интерфейсе для проверки не годится — она показывает только ваши задачи, а мешает как раз чужая. Дождитесь завершения или запустите с `--force`, решив, что этих задач не жалко.

«**Ваш docker-compose.yml отличается от файла версии**». В файле есть ваши правки — имя проекта, порты, лимиты, свои тома, — и замена унесла бы их молча. Скрипт печатает начало `diff` на экран, а полный список различий кладёт рядом в `compose-diff-<дата>.patch` и называет путь: переносить правки следует по файлу, а не по прокрутке терминала — на экран помещается версии две правок, не больше. Перенесите их в `new/docker-compose.yml` (там уже лежит файл целевой версии) и запустите `./update.sh $VER --compose-merged`.

Эта остановка повторится и на следующем обновлении — так и задумано. Эталоном для сравнения служит **чистый** файл версии, а не то, что было развёрнуто, поэтому ваши правки скрипт видит каждый раз и каждый раз просит перенести их в новый файл — вместо того чтобы однажды счесть их частью версии и молча стереть.

Отказ здесь тихий, поэтому и остановка громкая: если менялось имя проекта (`name:` в первой строке), после замены `docker compose ps` вернёт пустой список — работающая система просто перестанет быть видна командам, а `up -d` поднимет рядом вторую, пустую установку. Проверено на стенде ровно так и вышло: ноль контейнеров в списке при живой системе.

«**Версия добавила переменные, значение которых выбираете вы**». Новые переменные версии скрипт делит надвое. Те, у которых в `compose`-файле **есть** умолчание и шаблон предлагает **ровно его**, он вписывает в `.env` сам — с пояснением из шаблона и пометкой, каким обновлением они добавлены. Поведение системы от этого не меняется ни на букву: эти значения действовали и без строк в файле, просто были не видны, а `.env` после обновления совпадает с `.env` свежей установки. Остановка остаётся там, где нужен выбор: переменная **без** умолчания (без значения система не поднимется) и переменная, у которой шаблон советует не то, что подставится, — оба значения скрипт называет. Впишите нужное в `.env` и запустите снова.

Прежде остановка была на любой новой переменной, а обойти её позволял ключ `--accept-defaults`. Себя она не оправдала: вписывать почти всегда требовалось ровно то, что и так подставится, — и ключ приучал проходить мимо списка не читая, включая тот единственный раз, когда он был по делу. Сам ключ принимается по-прежнему и ничего не делает: сохранённая где-то команда с ним не должна падать на ровном месте.

«**Не удалось спросить базу о незавершённых фоновых задачах**». Контейнер `postgres` не отвечает или `POSTGRES_USER / POSTGRES_DB` в `.env` не те, что у него. Скрипт не делает вид, что проверил: пустой ответ на неудавшемся запросе выглядит ровно как «задач нет», и обновление снесло бы чужую сборку комплекта, отчитавшись об успешной проверке. Обойти — `--force`.

«**В new/ лежат файлы версии X, а обновляемся на Y**». `--compose-merged` разворачивает то, что лежит в `new/`, а там остались файлы прошлого обновления: развернув их, вы потеряли бы всё, что промежуточные версии добавили в `compose`. Запустите без ключа — скрипт скачает файлы нужной версии, покажет ваши правки, и перенесёте вы их уже в свежий файл.

«**docker compose не видит ни одного контейнера этой установки**». Либо запуск не из того каталога, либо в `docker-compose.yml` менялось имя проекта, а контейнеры подняты под прежним. Система при этом работает — просто командам не видна. Сверьте `docker compose ps` и `docker ps`.

«**Не удалось снять дамп базы**». Кончилось место или `pg_dump` отказал. Обновление не начиналось. Дамп — не формальность: схема мигрирует при старте новой версии, отката миграций нет, и вернуть прошлую схему можно только из него. Если копию вы сняли другим способом — `--no-backup`.

«**В релизе другой update.sh**». Скрипт версии N обновляет на N+k, и целевой выпуск может знать о шаге, которого сегодняшний скрипт не делает. Скрипт печатает готовую команду `curl` для обновления самого себя — выполните её и повторите запуск.

«**Обновление не подтвердилось**». Единственный случай, когда система уже переключена: контейнеры пересозданы, но `/api/version` не ответил целевой версией. Скрипт печатает последние 50 строк журнала `api` — причина почти всегда там: не хватило переменной окружения или оборвалась миграция. Дальше — откат (§8.3).

8.3. Откат

```
./update.sh --rollback
```

Возвращает сохранённые `docker-compose.yml.prev-*` и `.env.prev-*` и ждёт, пока система ответит прежней версией. Остановленные контейнеры откату не помеха — кроме базы: её скрипт спрашивает о миграциях и без ответа отказывается что-либо возвращать (`docker compose up -d postgres`, затем повторить).

Работает, только пока миграции не применились. Скрипт проверяет это сам: сравнивает список миграций в базе со снимком `migrations.prev-<дата>`, снятым перед обновлением, и, увидев новые, отказывает — старый образ с новой схемой работать не обязан. Если спросить базу не удалось или снимка нет (обновляли не скриптом), он тоже отказывает и объясняет, что смотреть: неизвестность здесь равна опасности, а цена ошибки — база. После миграций откат делается **только** восстановлением дампа (§9), причём базу перед этим **пересоздают**: дамп, залитый поверх мигрировавшей схемы, оставит смесь новой схемы со старыми данными. Другого пути нет — поэтому дамп из шага 3 и не обсуждается.

Миграции БД применяются автоматически при старте `api`. Данные лежат в томах (`postgres_data`, `minio_data`, `ollama_data`), и обновление их не трогает.

8.4. Разовые оговорки

Скрипт печатает только те, чей рубеж ваше обновление **пересекает**: 0.90.0 → 0.92.0 получит оговорку про 0.91.0 и промолчит про 0.137.0 и 0.139.0 — до них ещё идти; 0.90.0 → 0.141.0 получит все три. Здесь они полностью.

Разово при обновлении с версии ниже 0.139.0 — про локальную Ollama. Она больше не поднимается по умолчанию (профиль Compose, issue #824). На уже работающей установке обновление при этом ничего не выключает: `up -d` контейнер отключённого профиля не трогает, Ollama продолжит работать как работала. Выбор делается один раз:

- **пользуетесь ею** — раскомментируйте в `.env` `COMPOSE_PROFILES=ollama` и `OLLAMA_MODEL=...` (§7). Сама по себе она никуда не денется — переживает и `up -d`, и `down`, и перезагрузку хоста, — но исчезнет навсегда в день, когда контейнер уберут: явным `rm`, чисткой `docker system prune`, переносом на другой сервер. Профиль выключен, и обратно она не поднимется — а обнаружится это не сразу и не там;

- **не пользуетесь** — уберите её и освободите память и диск: `docker compose -f deploy/docker-compose.yml rm -sf ollama ollama-init` — именно `rm -sf`, потому что обычный `down` контейнер отключённого профиля не забирает (проверено). И проверьте **Настройка системы** → **Настройки** → **Поиск и распознавание**: на установке, где настройки хоть раз сохраняли, выбранная модель хранится в базе и очистку `.env` переживает — движок надо выключить там, иначе мониторинг будет вечно сообщать, что Ollama не отвечает.

Том `ollama_data` в обоих случаях цел: модель заново не качается. Проверено обновлением стенда `0.137.0` → `0.140.0`: контейнер `ollama` после `up -d` остался работать как работал.

Разово при переходе на поставку образами (с версии ниже 0.137.0). Прежде `api` и `web` собирались на хосте из исходников (`up -d --build`), теперь тянутся из GHCR. Тома, имена сервисов и данные те же; достаточно задать `APP_VERSION` в `.env` и выполнить `docker compose pull && docker compose up -d`. Локально собранные образы после этого можно удалить (`docker image prune`). Обратный путь — сборка из исходников — остаётся: `-f deploy/docker-compose.yml -f deploy/docker-compose.build.yml up -d --build`.

Разово при обновлении с версии ниже 0.91.0. Контейнеры перестали работать от `root`. Том `dp_keys` (ключи шифрования одноразовых ссылок) создавался от `root`, и владелец у существующего тома сам не поменяется — сервер сможет читать ключи, но не сможет выписать очередной, и через несколько недель ссылки сброса пароля начнут отказывать. Выполните один раз (`--entrypoint chown` обязателен: без него аргументы достанутся серверу приложений, он запустится от `root` — и ничего не поменяет, притом без единой ошибки):

```
docker compose -f deploy/docker-compose.yml run --rm --user root --entrypoint chown api -R app:app /app/dp-keys
```

На новой установке делать ничего не нужно: пустой том наследует владельца из образа.

Обновление **вручную**, шаг за шагом — **Приложение Б**. Оно остаётся полноценным путём: скрипту нужен доступ к `github.com` с хоста, и на закрытом контуре его просто нечем накормить.

8.5. Две команды рядом с обновлением

```
./update.sh --check      # есть ли выпуск новее вашего; ничего не меняет
./update.sh --gc         # какие старые образы можно убрать (удалить — с --yes)
```

`--check` сравнивает `APP_VERSION` из вашего `.env` с последним выпуском и отвечает **кодом выхода**: `0` — обновление есть, `3` — вы на последней версии, `1` — спросить не удалось. Docker ей не нужен, ничего не скачивается. Ответ кодами — чтобы её можно было поставить в `cron` или в мониторинг: «не смогли спросить» и «обновлений нет» для скрипта обязаны различаться, иначе оборвавшаяся сеть неделями выглядит как «всё свежее».

```
# По понедельникам в 9 утра — письмо, если вышла новая версия
0 9 * * 1 cd /opt/bhs-crg && ./update.sh --check
```

Письмо придёт именно тогда, когда есть новость: `cron` отправляет его на **любой** вывод, поэтому строку «новее ничего нет» команда печатает только человеку в терминал, а в `cron` молчит — ответ там несёт код выхода. Еженедельное «всё по-старому» приучило бы не читать эти письма, включая то единственное, где сказано про выпуск.

`--gc` показывает образы **BHS.CRG**, которые на хосте есть, а текущей версии не нужны: каждое обновление оставляет прежние `api` и `web`, вместе это под гигабайт, и сами они не исчезают. Без `--yes` команда только печатает список с размерами. Сторонние образы (`postgres`, `garage`, `ollama`) она не трогает — в отличие от `docker image prune -a`, который снёс бы и их, а платить за это пришлось бы простым на следующем старте. Наша **копия** MinIO в `ghcr.io` (§3) — тоже сторонний образ: отбор идёт по имени `bhs.crg-*`, а не по адресу реестра, иначе замороженный тег хранилища никогда не совпадал бы с `APP_VERSION` и попадал бы в список на удаление. Образ, на который ссылается хоть один контейнер — пусть даже остановленный, — в список не попадает: остановленный контейнер прежней версии это готовый откат. Без `.env` или без `APP_VERSION` в нём команда отказывается работать: не зная развёрнутой версии, она сочла бы ненужным **каждый** образ, включая тот, на котором система работает прямо сейчас.

8.6. Переход на PostgreSQL 18 (разово, при обновлении до 0.158.0)

Выпуск **0.158.0** переводит систему с PostgreSQL 16 на 18. Это единственный рубеж, который `update.sh` **отказывается** переходить сам: он остановится и покажет этот раздел.

Почему одним обновлением нельзя. Мажорная версия PostgreSQL не читает кластер предыдущей — это свойство самой СУБД. Вдобавок официальный образ с 18-й версии сменил раскладку: данные лежат в подкаталоге с номером мажора (`/var/lib/postgresql/18/docker`), а томом объявлен `/var/lib/postgresql`, а не `/var/lib/postgresql/data`.

Что будет, если обновиться не глядя (проверено на живых контейнерах, не предположение): контейнер базы **не запустится** — образ 18 увидит в томе кластер 16, откажется его трогать и напишет почему. Данные целы, система лежит. Вернуть её: положить обратно прежние `docker-compose.yml` и `.env` (скрипт сохраняет их как `docker-compose.yml.prev-*` и `.env.prev-*`) и поднять стек.

Отказ громкий именно потому, что **имя тома не менялось** — новый сервер натывается на старый кластер. Страховка здесь не том, а **дамп**: том придётся удалить, поэтому шаг с проверкой дампа пропускать нельзя.

Процедура выполняется у консоли, занимает минуты; система на это время недоступна.

```

cd /opt/bhs-crg                                # каталог установки
PGU=$(grep -E '^POSTGRES_USER=' .env | cut -d= -f2)
PGD=$(grep -E '^POSTGRES_DB=' .env | cut -d= -f2)

# 1. Дамп с РАБОТАЮЩЕЙ базы 16 – до всякого обновления.
mkdir -p backups
docker compose exec -T postgres pg_dump -U "$PGU" -d "$PGD" -Fc > backups/pre-pg18.dump

# 2. ПРОВЕРИТЬ дамп. Дальше он единственная копия данных: том мы удалим.
ls -lh backups/pre-pg18.dump                    # размер не должен быть нулевым
docker run --rm -i -v "$PWD/backups:/b" postgres:18.6-alpine pg_restore --list /b/pre-
pg18.dump | head                                # читается – значит дамп целый

# 3. Остановить систему.
docker compose down

# 4. Забрать docker-compose.yml выпуска, СОХРАНИВ свой.
cp docker-compose.yml docker-compose.yml.pre-pg18
curl -fsSL -o docker-compose.yml
https://github.com/pavru/bhs.crg/releases/download/v0.158.0/docker-compose.yml
rm -f docker-compose.yml.release                # эталон устарел; следующий update.sh
скачает свой
sed -i 's/^APP_VERSION=.* /APP_VERSION=0.158.0/' .env

```

Если в `docker-compose.yml` у вас были свои правки (порты, лимиты, `name:`), перенесите их в новый файл — сравните с `docker-compose.yml.pre-pg18`. Пропустив это, вы потеряете их молча: здесь нет проверки, которая ловит такое при обычном обновлении.

```

# 5. Удалить том прежнего кластера. Имя – с префиксом проекта ('name:' в compose).
docker volume rm bhs-crg_postgres_data

# 6. Поднять ТОЛЬКО базу и дождаться готовности (не одной командой pg_isready: пока идёт
#   инициализация, к временному серверу уже можно подключиться, а нужной базы ещё нет).
docker compose up -d --wait postgres
until docker compose exec -T postgres psql -U "$PGU" -tAc          "select 1 from pg_database
where datname='$PGD'" | grep -q 1; do sleep 1; done

# 7. Восстановить дамп.
docker compose exec -T postgres pg_restore -U "$PGU" -d "$PGD" --no-owner < backups/pre-
pg18.dump

# 8. Поднять остальное.
docker compose up -d

```

Порядок шагов 6–8 обязателен: подними вы весь стек сразу, сервер приложений увидел бы пустую базу, накати́л бы на неё миграции — и восстановление упёрлось бы в уже существующие таблицы.

Проверка после перехода:

```
docker compose exec -T postgres psql -U "$PGU" -tAc 'show server_version'      # 18.x
docker compose exec -T postgres psql -U "$PGU" -d "$PGD" -tAc 'select count(*) from
"__EFMigrationsHistory" '                                                    # не ноль
```

Затем откройте систему и убедитесь, что комплекты, каталог и документы качества на месте. Дамп `backups/pre-pg18.dump` сохраните до тех пор, пока не будете уверены: пока он есть, откат к 16 возможен (прежний `docker-compose.yml.pre-pg18` + восстановление дампа в новый том 16).

8.7. Переход на Garage (разово, при обновлении до 0.160.0)

Выпуск **0.160.0** переводит систему с MinIO на Garage. В отличие от перехода на PostgreSQL 18 (§8.6), делать руками ничего не нужно: `update.sh` переносит файлы сам, внутри обновления. Раздел описывает, что при этом происходит, — чтобы происходящее на экране не выглядело неожиданностью.

Почему меняем хранилище. Репозиторий MinIO архивирован разработчиком 13.02.2026: community-версия больше не собирается и исправлений безопасности не получает. Образ последнего выпуска хранится в нашем реестре и никуда не денется (он нужен как раз для этого перехода), но оставаться на незакрываемых уязвимостях — не решение.

Что делает обновление (`./update.sh 0.160.0`), по шагам:

1. Считает объём файлов и свободное место. Не хватает — останавливается **до** любых изменений: во время перехода файлы какое-то время лежат в обоих хранилищах.
2. Создаёт ключи нового хранилища (`GARAGE_KEY_ID` , `GARAGE_SECRET` , `GARAGE_RPC_SECRET` , `GARAGE_ADMIN_TOKEN`) и вписывает их в `.env` . Придумывать их не нужно: формат жёсткий, и значение неверной длины хранилище просто не примет. Имя `bucket` переносится из `MINIO_BUCKET` — поэтому пути к файлам в базе не меняются.
3. **Останавливает приложение** (`api` , `web`) на время копирования. Это единственный простой в обновлении, и он неизбежен: файл, записанный после того как копирование прошло его каталог, в новое хранилище не попал бы — молча.
4. Поднимает Garage, готовит его (раскладка, ключ, `bucket`, права) и копирует файлы.
5. **Сверяет**: состав и размеры объектов в обоих хранилищах должны совпасть. Не сошлось — обновление останавливается, прежние файлы возвращаются, система поднимается на прежней версии.
6. Только после успешной сверки подменяет `docker-compose.yml` , `.env` и поднимает систему.

Прежнее хранилище не удаляется. Том `minio_data` остаётся на диске со всеми файлами на момент перехода — это страховка и точка отката. Удалять его стоит не раньше, чем система проработает на новом хранилище достаточно, чтобы вы были уверены; занимаемое место видно в `docker system df -v` .

Откат. `./update.sh --rollback` работает и через этот рубеж: перед возвратом прежней версии он копирует файлы **обратно** в MinIO — включая те, что появились уже после перехода. Если хранилище Garage при этом недоступно, откат отказывается работать и говорит почему: вернуть систему, потеряв

файлы последних часов, хуже, чем не вернуть вовсе.

Сколько это занимает. Копирование идёт со скоростью локального диска: на 320 МБ и 880 файлах — около двух секунд (замерено). Простой приложения примерно равен времени копирования плюс полминуты на подъём нового хранилища.

Если перенос прервался. Ничего страшного: прежнее хранилище не изменяется вовсе. Повторите `./update.sh 0.160.0` — копирование продолжится, уже перенесённые файлы не копируются заново.

9. Резервное копирование и восстановление

База данных:

```
# Бэкап
docker compose -f deploy/docker-compose.yml exec -T postgres \
  pg_dump -U postgres --clean --if-exists bhs_crg > backup_$(date +%F).sql

# Восстановление. База ПЕРЕСОЗДАЁТСЯ, а не заливается поверх, и это проверено: дамп, наложенный
# на схему, которую уже изменили миграции новой версии, восстанавливает свои объекты и молча
# оставляет чужие — таблицы, добавленные новой версией, остаются, а список применённых миграций
# возвращается к старому. Ошибок при этом ноль, выглядит как успех, а следующий старт api
# упирается
# в попытку применить миграцию поверх уже существующих таблиц.
docker compose -f deploy/docker-compose.yml stop api
docker compose -f deploy/docker-compose.yml exec -T postgres psql -U postgres -d postgres \
  -c 'DROP DATABASE bhs_crg WITH (FORCE);' -c 'CREATE DATABASE bhs_crg OWNER postgres;'
cat backup_YYYY-MM-DD.sql | docker compose -f deploy/docker-compose.yml exec -T postgres \
  psql -U postgres -d bhs_crg
docker compose -f deploy/docker-compose.yml start api
```

Порядок важен: api останавливают **до** пересоздания базы и запускают **после**. Миграции он применяет при старте — подключившись к восстановленной базе, версия новее той, из которой снята копия, тут же мигрирует её обратно вперёд, и восстановление окажется отменено. Поэтому, восстанавливаясь после неудачного обновления, сперва верните в `.env` прежний `APP_VERSION`.

Файлы (MinIO): резервируйте том `minio_data` (или зеркалируйте bucket через `mc mirror`).

Копия средствами самой системы

Настройка системы → Настройки → Резервное копирование — кнопка «Создать копию». Копия ложится в каталог `BACKUP_DIR` на хосте (\$4) и оттуда же восстанавливается; через браузер при этом не передаётся ничего, кроме имени файла, поэтому размер копии перестал упираться в пределы транспорта.

Снятие идёт **фоновой задачей**: копия несёт библиотеку документов качества со сканами, и на рабочей системе это минуты. Ход виден пилюлей задач в шапке, итог приходит в колокольчик — вкладку можно закрыть.

Состав копии выбирается при снятии:

- **только настройка** — типы документов, шаблоны и их ассеты, справочники, общие данные, библиотека Typst, профили распознавания, шаблоны маппинга и рецепты обработки, алиасы сверки и библиотека документов качества со сканами;
- **настройка и проектные данные** — плюс стройки, разделы, комплекты, документы с выпущенными PDF, наборы данных с разобранными источниками, определения сверок и связки с материалами. Это и есть копия для восстановления после сбоя и для переезда: развёрнутая на чистой установке, она даёт рабочие стройки и комплекты без `pg_dump` и без доступа к хосту.

Плановые копии (расписание ниже) снимаются **полными**: их снимают не ради страховки настройки, а ради того дня, когда восстанавливать придётся всё.

Не входит ни в какой состав: учётные записи, ключи интеграций и пароль почты, результаты прогонов и собранный PDF комплекта. Дамп базы и том `minio_data` выше остаются нужны там, где важно восстановить систему **побайтно**, вместе с пользователями и историей.

Перенос на другой сервер. Скопируйте файл копии в `BACKUP_DIR` целевой установки — и всё:

```
rsync -av backups/crg-backup-20260822-101500-v0.141.0.zip user@target:/opt/bhs-crg/backups/
```

Файл появится в списке на целевой системе (список читает каталог, а не базу) и восстанавливается оттуда же. Ни правки `.env`, ни пересоздания контейнеров это не требует. Паспорт копии — дата, версия системы, состав — лежит **внутри** архива, поэтому переносить нужно один файл, а не пару.

Копия, снятая версией до 0.141.0, паспорта не несёт: в списке она видна, состав у неё показан не будет, восстановить её можно.

Права нужны на **каталог**, а не на сам файл: удаление из интерфейса опирается на право записи в каталог (он принадлежит `app`, см. §4), поэтому копия, положенная от `root`, и читается, и удаляется без правки владельца.

Скачивание и загрузка через браузер остаются удобством для небольших копий: файл проходит через память вкладки, и копию в гигабайты так не переносят — для неё есть путь выше. Предел загрузки задаётся `BACKUP_MAX_ARCHIVE_MB` (§4), и отказ по нему прямо называет этот выход.

Плановые копии и вынос их с сервера

Копии снимаются **автоматически**: расписание включено по умолчанию (раз в сутки в 03:00, хранить семь) и настраивается на том же экране. Настраивать при развёртывании нечего — но знать про два разных числа стоит: `BACKUP_KEEP_COUNT` (§4) — вместимость каталога, то есть про диск; «хранить плановых» в расписании — сколько из них убирать автоматически. Второе не может быть больше первого, а разница между ними и есть место под ручные копии.

Копия на том же диске — не защита от отказа диска. Она защищает от ошибки человека, неудачного обновления и порчи данных — то есть от большинства бед, — но не от того, что перестанет существовать сам носитель. Пока `BACKUP_DIR` лежит на том же диске, что и база, у вас нет резервной копии в том смысле, в каком это слово обычно понимают.

Вынос копий наружу система на себя не берёт намеренно: у каждой организации это своё хранилище, свои ключи и свои правила доступа, и приложение, которое ходит на чужой NAS или в S3, требует хранить у себя учётные данные к месту, куда складывается вся система целиком. Делается это средствами хоста, парой строк в `cron` :

```
# Ежедневно в 04:30 – после того, как отработало плановое копирование системы (03:00).
30 4 * * * rsync -a --delete /opt/bhs-crg/backups/ backup@nas:/volume1/bhs-crg/

# Или в объектное хранилище (S3, Ceph, Selectel, Яндекс) – rclone настраивается один раз:
30 4 * * * rclone sync /opt/bhs-crg/backups/ remote:bhs-crg-backups
```

Время выбирают с запасом после планового: копия в несколько гигабайт снимается минутами, а `rsync` недописанного архива увёз бы половину файла. Незавершённые копии система пишет во временные файлы с точкой в начале имени — их можно исключить (`--exclude='.*'`), но и без этого они не помешают: готовая копия появляется в каталоге одним движением, под своим окончательным именем.

Восстановление проверяют, а не предполагают. Копия, которую ни разу не разворачивали, — предположение о том, что она годная. Раз в квартал разверните стенд (§3) на отдельной машине или в отдельном каталоге, положите в его `BACKUP_DIR` свежую копию и восстановитесь из неё. Это единственный способ узнать, что механизм работает, в день, когда это ещё не срочно.

10. Диагностика

Симптом	Причина / решение
<code>docker compose</code> отвечает <code>'compose' is not a docker command</code>	Docker поставлен без плагина Compose v2 — обычно это пакет <code>docker.io</code> из репозитория дистрибутива. Переставьте из официального репозитория: Приложение А
Установка «ждёт готовности», а система работает	На сервере задан прокси (<code>all_proxy</code> , <code>http_proxy</code>) — и curl уводит через него даже обращение к самому себе. Проверить: <code>env grep -i proxy</code> . Начиная с 0.163.0 скрипты обращаются к петле мимо прокси; на более старых — обновите <code>install.sh</code> / <code>update.sh</code> со страницы выпуска
<code>api</code> падает при старте	Обязательное значение не задано или оставлено из примера — сообщение в лог называется, какое именно и какой переменной задаётся: <code>JWT_KEY</code> , <code>GARAGE_KEY_ID</code> , <code>GARAGE_SECRET</code> , <code>GARAGE_BUCKET</code> , параметры БД. Сюда же попадает <code>MODULES_ENABLED</code> с незнакомым кодом модуля: сообщение перечислит, какие коды в сборке есть. Проверьте также доступность <code>postgres</code> (healthcheck). <code>docker compose logs api</code>
Генерация PDF с ошибкой шрифтов	В образ включены DejaVu/Liberation/Noto. Если шаблон требует особый шрифт — добавьте его в <code>Dockerfile.api</code>
Распознавание не работает	Локальная Ollama по умолчанию не разворачивается — включается профилем (§7). Иначе: модель не загружена (<code>ollama pull</code>) или не задан API-ключ облачного движка
В состоянии системы «Ollama (распознавание): недоступна», хотя локальное распознавание не включали	Приложение считает движок включённым, а контейнера на хосте нет. Либо в <code>.env</code> задан <code>OLLAMA_MODEL</code> без <code>COMPOSE_PROFILES=ollama</code> , либо модель сохранена в базе из интерфейса (она сильнее переменной). Задайте профиль — или выключите движок в Настройки → Поиск и распознавание
Раскомментировал <code>COMPOSE_PROFILES=ollama</code> , <code>up -d</code> без ошибок, а контейнеров <code>ollama</code> нет	Сборка Compose не читает эту переменную из <code>.env</code> (ранние выпуски v2 — только из окружения). Тот же результат даёт флаг: <code>docker compose --profile ollama up -d</code>
Профиль <code>ollama</code> включён, контейнер работает, а распознавание его не берёт	Не задан <code>OLLAMA_MODEL</code> : движок без выбранной модели в перебор не попадает. В настройках он помечен «не выбрана модель», то же скажет <code>docker compose logs ollama-init</code>
Убрал <code>COMPOSE_PROFILES=ollama</code> , а контейнер <code>ollama</code> всё работает	Так и задумано Compose: профиль решает, что поднимать, а не что останавливать — <code>up -d</code> (и <code>--remove-orphans</code>) запущенный контейнер не трогает. Уберите явно: <code>docker compose rm -sf ollama ollama-init</code>
В состоянии системы компонент «отвечает через раз» (янтарная точка)	Проба доходит не всегда, но отказ ещё не объявлен — движку на это даётся три проверки подряд. Обычная причина не в поставщике, а в пути к нему: у имени несколько адресов, и часть из них недостижима. Проверить с сервера: <code>curl --ipv4</code> против <code>curl --ipv6</code> на адрес поставщика — если различие в секундах, дело в маршруте. Приложение с 0.165.0 пробует адреса внахлёт и такой путь обходит; если «через раз» держится, недостижимы все адреса одного семейства

Симптом	Причина / решение
Веб-интерфейс открывается, но «не видит» сервер	Проверьте, что сервис <code>web</code> проксирует на <code>api:8080</code> (см. <code>deploy/nginx.conf</code>)
Вход отвечает «слишком много попыток входа»	Сработал предел частоты по адресу клиента. Если за одним внешним адресом работает целый офис — так и задумано. Если предел ловят единичные пользователи, значит адрес клиента до сервера не доходит: проверьте, что каждый прокси в цепочке дописывает <code>X-Forwarded-For</code> (в поставляемом <code>nginx.conf</code> это <code>\$proxy_add_x_forwarded_for</code>), а сети всех прокси попадают в <code>FORWARDED_TRUSTED_NETWORKS</code> — иначе адресом клиента станет прокси, один на всех
Ссылки сброса пароля перестали работать после обновления	Права на том <code>dp_keys</code> — см. разовую команду в §8.4
<code>api</code> или <code>ollama</code> перезапускаются без внятной причины, в логах обрыв на тяжёлой операции	Упёрлись в потолок памяти контейнера. Поднимите <code>API_MEMORY_LIMIT</code> / <code>OLLAMA_MEMORY_LIMIT</code> в <code>.env</code> . Проверить: <code>docker stats</code> и <code>docker inspect --format '{{.State.OOMKilled}}' <контейнер></code>
Вёрстка PDF отвечает «не завершилась за 60 с»	Шаблон зациклился либо рекурсия без выхода. Процесс останавливается намеренно: без срока он занимал бы ядро до перезапуска сервера
Не загружаются крупные файлы	Наибольший предел — архив резервной копии, и задаётся он одной переменной <code>BACKUP_MAX_ARCHIVE_MB</code> : из неё же считается <code>client_max_body_size</code> при старте <code>web</code> (см. <code>deploy/nginx-body-size.sh</code>). Два числа в разных файлах здесь держать нельзя — порог <code>nginx</code> ниже нашего делает предел приложения недостижимым, а отказ приходит HTML-страницей <code>nginx</code> без поля <code>error</code> , и интерфейс показывает голое сообщение <code>axios</code> . Проверить действующее значение: <code>docker compose logs web grep client_max_body_size</code>
Восстановление отвечает «Файл превышает N МБ»	Копия переросла предел. Текущий вес против предела виден в интерфейсе: Настройки → Резервное копирование . Поднимите <code>BACKUP_MAX_ARCHIVE_MB</code> и пересоздайте <code>api</code> и <code>web</code>

Полезные команды:

```
docker compose -f deploy/docker-compose.yml ps          # статус
docker compose -f deploy/docker-compose.yml logs -f api # логи сервера
docker compose -f deploy/docker-compose.yml down        # остановить (тома сохраняются)
docker compose -f deploy/docker-compose.yml down -v     # остановить и УДАЛИТЬ данные
```

11. Безопасность (чек-лист перед эксплуатацией)

- ☐ Сменены пароли `POSTGRES_PASSWORD`, `MINIO_ROOT_PASSWORD`, а также `MINIO_ROOT_USER` (значение из примера — `minioadmin`).
- ☐ Задан свой `JWT_KEY` (≥ 32 символов, случайный). Без этого приложение не запустится.

- ☐ Заданы свои `MINIO_ROOT_USER` / `MINIO_ROOT_PASSWORD`, `MINIO_BUCKET` и все параметры БД (`POSTGRES_DB` / `POSTGRES_USER` / `POSTGRES_PASSWORD`). Значения из файла-примера приложение тоже не примет — запуск останавливается с указанием переменной.
- ☐ Порты БД/MinIO/Ollama не опубликованы в интернет. Консоль MinIO поставляемый `docker-compose.yml` публикует только на петлю — убедитесь, что `MINIO_CONSOLE_BIND` не переопределён на `0.0.0.0` (см. §6).
- ☐ Том `dp_keys` защищён как секрет: доступ к нему (или к его копии) позволяет выписать действительную ссылку сброса пароля на любую учётную запись, минуя пароль и блокировку. В общие резервные копии его не кладут; если кладут — хранят вместе с прочими секретами. **Он же расшифровывает ключи интеграций** (см. §7): при потере тома ключи распознавания, веб-поиска и пароль SMTP придётся ввести заново — данные и документы при этом не страдают.
- ☐ **HTTPS обязателен**, если система доступна не только с локальной машины: перед `web` стоит обратный прокси с терминацией TLS. Порт `80` при этом остаётся открытым — по нему идут проверка Let's Encrypt и перенаправление на HTTPS, — но самого приложения на нём нет. Приложение TLS не завершает и само на HTTPS не перенаправляет — без внешнего контура пароли и токены идут открытым текстом. Порядок установки — **Приложение В**.
- ☐ После установки прокси: в `.env` задан `WEB_BIND=127.0.0.1`, и порт `WEB_PORT` снаружи не отвечает. Иначе рядом с HTTPS остаётся открытый вход по HTTP, в обход всего контура.
- ☐ Создан администратор; лишние тестовые учётные записи удалены.
- ☐ Настроено регулярное резервное копирование БД и `minio_data`.
- ☐ Известно, где смотреть вес встроенной резервной копии против предела восстановления (**Настройки** → **Резервное копирование**). Вес задаётся библиотекой документов качества и растёт вместе с ней; предел поднимается переменной `BACKUP_MAX_ARCHIVE_MB`.

Приложение А. Установка Docker

§2 требует **Docker Engine 24+** и **Compose v2 или новее**, но на чистом сервере их ещё нет. Здесь — путь от голой ОС до состояния, в котором «Быстрый старт» (§3) проходит целиком.

Ставим **только из официального репозитория Docker**: так одной установкой получаются и свежий движок, и `docker compose` как его плагин. Пакеты из репозитория дистрибутивов и `snar` тоже называются «докер», но требованиям §2 не отвечают — чем именно это кончается, в А.4.

Команды ниже прогнаны на чистых Ubuntu 24.04 и Rocky Linux 9 (август 2026): из репозитория приходят Engine 29.x и плагин Compose 5.x. Официальная страница установки со временем меняется (формат файла репозитория здесь уже менялся) — если команда не сработала, сверьтесь с <https://docs.docker.com/engine/install/>.

А.1. Ubuntu 22.04+ / Debian 12+

Порядок один и тот же, отличается одна переменная в начале.

```

DISTRO=ubuntu          # для Debian: DISTRO=debian

# 1. Снять пакеты, которые займут имена команд и будут мешать.
#   Часть из них в системе не стоит — сообщения «не найден» здесь ожидаемы.
for p in docker.io docker-doc docker-compose docker-compose-v2 \
        docker-buildx podman-docker containerd runc; do
    sudo apt remove -y "$p" 2>/dev/null || true
done

# 2. Ключ и репозиторий Docker
sudo apt update
sudo apt install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL "https://download.docker.com/linux/$DISTRO/gpg" -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

sudo tee /etc/apt/sources.list.d/docker.sources >/dev/null <<EOF
Types: deb
URIs: https://download.docker.com/linux/$DISTRO
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF

# 3. Установка
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io \
                    docker-buildx-plugin docker-compose-plugin

```

Подстановки внутри <<EOF выполняет ваша оболочка, а не `sudo`: в файл попадают уже готовые кодовое имя выпуска и архитектура. Стоит на них взглянуть — `cat /etc/apt/sources.list.d/docker.sources`. Пустое `Suites`: означает, что `/etc/os-release` не назвал кодовое имя (бывает на производных сборках); подставьте его вручную — `jammy`, `noble`, `bookworm`.

Дальше — A.3.

A.2. RHEL-семейство: Rocky Linux 9 / AlmaLinux 9 / CentOS Stream 9

```

sudo dnf -y install dnf-plugins-core
sudo dnf config-manager --add-repo https://download.docker.com/linux/centos/docker-ce.repo
sudo dnf -y install docker-ce docker-ce-cli containerd.io \
                    docker-buildx-plugin docker-compose-plugin

```

На самой **RHEL** адрес другой — `https://download.docker.com/linux/rhel/docker-ce.repo`. Rocky и Alma собираются из исходников RHEL, но отдельного репозитория у Docker для них нет: им предназначен именно `centos`.

firewalld. В RHEL-семействе он включён по умолчанию — и опубликованные Docker порты он **не закрывает**: правила Docker стоят раньше. С `ufw` в Ubuntu ровно так же. Ограничивать доступ нужно там, где порт публикуется: привязкой к адресу в `docker-compose.yml` (как сделано для консоли MinIO, см. §6) или фильтром периметра. И проверять снаружи, а не выводом `firewall-cmd --list-ports`: список честно покажет, что порт не открыт, а `curl` с соседней машины в это же время откроет страницу.

SELinux трогать не нужно: поставка работает с именованными томами, метки для них расставляются сами. Понадобится, только если добавите свой `bind-mount`, — тогда ему нужен суффикс `:z` в описании тома.

А.3. После установки — оба варианта

```
# На Debian/Ubuntu службу запустил и включил сам пакет — команда ничего не изменит.
sudo systemctl enable --now docker

# Чтобы не писать sudo перед каждой командой (о цене — сразу под блоком).
sudo usermod -aG docker "$USER"
newgrp docker                                # применить группу без перелога (в этой оболочке)
```

 **Группа `docker` — это `root` на хосте**, без оговорок: её участник запускает контейнер, смонтировав в него корень файловой системы, и дальше волен делать что угодно. Не «почти `root`» и не «`root` внутри контейнера» — доступ ко всему серверу. Включайте в неё только тех, кому и так дали бы `sudo` без пароля; на сервере с несколькими пользователями разумнее оставить `sudo docker ...`. (У Docker есть `rootless`-режим, снимающий это ограничение; наша поставка на нём не проверялась.)

Если сервер выходит в интернет через прокси, задать его нужно демону: переменные вашей оболочки Docker не наследует, и без этого `docker compose up` останавливается на скачивании образов, жалуясь на таймаут — то есть на что угодно, только не на прокси.

```
sudo mkdir -p /etc/systemd/system/docker.service.d
sudo tee /etc/systemd/system/docker.service.d/http-proxy.conf >/dev/null <<'EOF'
[Service]
Environment="HTTP_PROXY=http://proxy.example:3128"
Environment="HTTPS_PROXY=http://proxy.example:3128"
Environment="NO_PROXY=localhost,127.0.0.1"
EOF
sudo systemctl daemon-reload && sudo systemctl restart docker
```

Это прокси **только для скачивания образов**. Сама система в интернет через него не пойдёт, и переменные `HTTP_PROXY / HTTPS_PROXY`, переданные в контейнер `ari`, она **намеренно не использует**: иначе в прокси ушли бы и хранилище, и Ollama, и плагины. При старте в журнале `ari` будет предупреждение, если такие переменные заданы. Прокси для внешних сервисов (распознавание, поиск, почта, проверка обновлений) задаётся в интерфейсе: **Настройки** → **Прокси**, с галкой «Через прокси» у каждого сервиса — см. `ADMIN_GUIDE.md`, раздел «Настройки».

Если прокси подменяет сертификаты (TLS-инспекция), корневой сертификат компании нужно добавить в контейнер `ari` (например, смонтировать файл и задать `SSL_CERT_FILE`), иначе внешние HTTPS-запросы будут отклоняться с ошибкой доверия. Если через прокси разрешена загрузка по внешним ссылкам, запретите на самом прокси обращения к внутренним адресам (RFC 1918, петля, 169.254.0.0/16).

A.4. Чего не ставить

Что	Чем это кончится
<code>apt install docker.io</code> — пакет из репозитория дистрибутива	Движка может хватить, а плагина Compose в нём нет: <code>docker compose version</code> отвечает <code>'compose' is not a docker command</code> . Половина команд этой инструкции просто не существует, причём выглядит это как опечатка в инструкции, а не как незаконченная установка
<code>docker-compose v1</code> (через <code>pip</code> или отдельным бинарником — с дефисом)	Другая программа: реализация на Python, снята с поддержки. Наш <code>docker-compose.yml</code> написан по спецификации Compose v2 (в нём намеренно нет ключа <code>version:</code>), и v1 такой файл не понимает — падает на разборе, называя незнакомые поля
snap-пакет <code>docker</code>	Работает в изоляции snap: свои пути к данным и запрет на чтение файлов вне домашнего каталога. Отказы приходят как «нет прав» или «файл не найден» — по ним причину не угадать

Общее у всех трёх: они дают ошибки, не похожие на свою причину. Если `docker` в системе уже стоял и вы не знаете откуда — проверьте `docker compose version` (плагин обязан отвечать) и `dpkg -l | grep -i docker / rpm -qa | grep -i docker`.

A.5. Проверка

```
docker --version          # 24.0 или новее (сегодня из репозитория ставится 29.x)
docker compose version    # плагин: через пробел, не через дефис
docker run --rm hello-world # запуск контейнера целиком, от скачивания до вывода
```

`hello-world` скачивается с Docker Hub, а образы системы — с `ghcr.io`. Успех этой команды означает «Docker работает», но не «до GHCR дотянемся»: это покажет уже `docker compose up`.

Дальше — **§3 «Быстрый старт»**.

Приложение Б. Обновление вручную

Тот же порядок, что делает `update.sh` (§8), но руками — для случая, когда скрипт неприменим: закрытый контур без доступа к `github.com`, нестандартная установка, желание видеть каждый шаг. Разовые оговорки при переходе через 0.91.0, 0.137.0 и 0.139.0 — в §8.4, они действуют и здесь.

Порядок ниже устроен так, что всё, что может отказать — загрузка образов, разбор нового `compose`-файла, недостающая переменная, — происходит **до** первой остановки. Дойдя до `up -d`, вы уже знаете, что обновление состоится.

Где выполнять. Команды написаны для установки из релиза: `docker-compose.yml` и `.env` лежат в текущем каталоге. Если система поставлена из клона репозитория, эти файлы — в `deploy/`: перейдите туда (`cd deploy`), и всё остальное совпадёт дословно.

Кого прервёт. Перезапуск обрывает фоновые задачи: незавершённая сборка комплекта или распознавание сканов сами не возобновятся, новая версия при старте пометит их неудавшимися. Пилуля задач в интерфейсе для проверки **не годится** — она показывает только ваши собственные задачи, а мешает как раз чужая. Спросить надо базу:

```
docker compose exec -T postgres psql -U postgres -d bhs_crg \  
-c "select \"Status\", count(*) from jobs where \"Status\" in ('Queued', 'Running') group by  
1;"
```

Пустой ответ — можно обновляться. (`postgres` и `bhs_crg` — значения `POSTGRES_USER` и `POSTGRES_DB` из вашего `.env`.)

Сколько займёт — те же 19 секунд со стенда, что и у скрипта (§8.1).

```

# 1. РЕЗЕРВНАЯ КОПИЯ – до всего остального (раздел 9). Схема базы мигрирует при старте новой
# версии, и отката миграций нет: вернуть прошлый образ можно, прошлую схему – только из
копии.

# 2. Выбрать версию: https://github.com/pavru/bhs.crg/releases
VER=0.140.0

# 3. Забрать файлы ИМЕННО ЭТОЙ версии, во временный каталог – не поверх своих. Compose-файл
# приглашает к правкам (лимит cpus, порты, свои тома), и `curl -LO` рядом с ним унёс бы их
молча.
mkdir -p new && cd new
curl -LO https://github.com/pavru/bhs.crg/releases/download/v$VER/docker-compose.yml
curl -Lo env.example https://github.com/pavru/bhs.crg/releases/download/v$VER/env.example
cd ..

# 4. Сохранить то, что работает сейчас: без этих копий откатываться будет не на что.
cp docker-compose.yml docker-compose.yml.prev
cp .env .env.prev

# 5. Сравнить и перенести. Новые версии добавляют переменные, тома и лимиты; обновив только
# образы, вы получите новую настройку в её умолчании и не узнаете об этом.
diff docker-compose.yml new/docker-compose.yml # слева ваши правки, справа изменения версии
diff .env.example new/env.example             # что добавилось в шаблон окружения
cp new/docker-compose.yml docker-compose.yml   # свои правки, если они были, вернуть руками
cp new/env.example .env.example

# 6. Указать версию и добавить то, что появилось в шаблоне
nano .env                                     # APP_VERSION=$VER

# 7. Предполётная проверка – ничего ещё не останавливая
docker compose config -q                     # молчание = файл разобран и APP_VERSION задан
docker compose pull                          # образы новой версии уже на хосте

# 8. Обновиться (в соседней консоли полезно держать `docker compose logs -f api`)
docker compose up -d

# 9. Проверить
docker compose ps                            # api и web – running, api – healthy
curl -s http://localhost:8080/api/version

```

Шаг 5 — про **ваши** правки, а не про новые строки версии. Отказ здесь тихий: заменив файл целиком, вы не получите ошибки. Если менялось, например, имя проекта (name: в первой строке), `docker compose ps` после замены вернёт пустой список — работающая система просто перестанет быть видна командам, а `up -d` поднимет рядом вторую, пустую установку. Проверено на стенде ровно так и вышло: ноль контейнеров в списке при живой системе.

Шаг 7 — не формальность, но и не полная проверка. `config -q` ручается за то, что `compose`-файл разобран и `APP_VERSION` задан (только эта переменная объявлена обязательной); пустые `JWT_KEY` или `POSTGRES_PASSWORD` он пропустит молча, они выяснятся уже при старте. `pull` же до `up -d` означает, что сорвавшаяся загрузка образа — недоступный `ghcr.io`, оборванная сеть — оставит систему работать на прежней версии. Оба отказа стоят ноль простоя, если случились здесь.

Анонимный ответ `/api/version` показывает только номер версии: хеш коммита и дату сборки система отдаёт вошедшему пользователю — они видны внизу боковой панели интерфейса (`v0.140.0 · a1b2c3d`, дата сборки в подсказке). Пустое поле `commit` в ответе `curl` — не признак неправильно собранного образа.

Миграции БД применяются автоматически при старте `api`. Данные сохраняются в томах (`postgres_data`, `minio_data`, `ollama_data`) и обновление их не трогает.

Если новая версия не поднялась.

```
docker compose ps -a          # кто не в running
docker compose logs --tail=100 api  # причина почти всегда в первых строках
```

`up -d` в этом случае не завершится быстро: `web` ждёт готовности `api` по `healthcheck`, а тот сдаётся не сразу — 20 секунд на старт плюс 30 попыток по 10 секунд, то есть до пяти с лишним минут молчания, прежде чем появится строка вида `dependency failed to start: container bhs-crg-api-1 is unhealthy`. Прерывать не нужно; ради этого и держат вторую консоль с `logs -f api` — там причина видна сразу. Пока `api` не отвечает, старая веб-часть продолжает работать и отдаёт `502` на любой запрос к API — сказать о причине ей нечем, причина в журнале `api`. Самые частые: не хватило переменной окружения, которую добавила новая версия (пропущен шаг 5), и оборвавшаяся миграция.

Откат. Пока миграции не применились (в журнале `api` нет строк `Applying migration`), достаточно вернуть сохранённое на шаге 4 и повторить обновление в обратную сторону:

```
cp docker-compose.yml.prev docker-compose.yml
cp .env.prev .env
docker compose up -d
```

Если миграции применились — старый образ с новой схемой работать не обязан, и откат делается только восстановлением резервной копии (раздел 9), причём **базу перед этим пересоздают**: дамп, залитый поверх мигрировавшей схемы, оставит смесь новой схемы со старыми данными. Другого пути нет — поэтому копия из шага 1 и не обсуждается.

Приложение В. HTTPS: nginx + Let's Encrypt

§2 требует HTTPS, §11 повторяет требование — здесь путь от работающего по HTTP стека до `https://`. Порядок рассчитан на Ubuntu/Debian; на RHEL-семействе отличаются только имена пакетов.

Зачем это не «для галочки». Кроме паролей и токенов, идущих открытым текстом, у HTTP есть следствие, которое видно сразу: браузер объявляет такую страницу **незащищённым контекстом** и отключает часть своих возможностей. В системе от этого не работают буфер обмена («Скопировать»), снимок экрана в форме сообщения об ошибке — и до версии 0.146.3 падали три экрана целиком (панель параметров шаблона, составные поля документа, редактор разбиения PDF). Падать перестало, но полноценно интерфейс работает только по HTTPS.

В.1. Что нужно до начала

- Доменное имя с **А-записью** на адрес сервера (`docs.example.ru` в примерах ниже). Let's Encrypt проверяет владение доменом обращением к нему же — на IP-адрес сертификат не выдаётся.
- Открытые снаружи **80** и **443**. Порт 80 нужен и после выпуска: по нему идёт продление.
- Работаящий стек (§3): `curl -s http://localhost:8080/api/version` отвечает номером версии.

В.2. nginx на хосте

Прокси ставится **на хост**, а не ещё одним контейнером Compose: продление сертификата и перезагрузка конфигурации тогда не завязаны на жизненный цикл стека, а `docker compose down` не уносит с собой веб-вход.

```
sudo apt update && sudo apt install -y nginx
sudo systemctl enable --now nginx
```

Если система ставилась скриптом с `--reverse-proxy`, готовый `reverse-proxy.conf` уже лежит в каталоге установки — с вашим именем, портом и пределом размера тела. Тогда скачивать и заполнять образец не нужно: копируйте свой файл (шаги про `ln -s`, `nginx -t` и `certbot` ниже — те же).

```
sudo cp /путь/к/установке/reverse-proxy.conf /etc/nginx/sites-available/bhs-crg
```

Команды с подставленным путём скрипт печатает в конце установки, и они же — в шапке самого файла.

Конфигурация — образец `reverse-proxy.conf.example`. Он **прикладывается к каждому выпуску начиная с 0.162.0**, поэтому берите его со страницы своего выпуска, а не из репозитория: файл в master может уйти вперёд установленной версии.

```
# Со страницы СВОЕГО выпуска: подставьте номер установленной версии (APP_VERSION из .env)
VER=0.162.0
curl -LO "https://github.com/pavru/bhs.crg/releases/download/v$VER/reverse-proxy.conf.example"

sudo cp reverse-proxy.conf.example /etc/nginx/sites-available/bhs-crg
sudo nano /etc/nginx/sites-available/bhs-crg      # заменить docs.example.ru; порт — WEB_PORT из
.env
sudo ln -s /etc/nginx/sites-available/bhs-crg /etc/nginx/sites-enabled/
sudo rm -f /etc/nginx/sites-enabled/default      # иначе стандартная страница nginx перехватит
имя
sudo nginx -t && sudo systemctl reload nginx
```

В образце один server-блок, и он на 80 — так устроен certbot --nginx : он находит блок с вашим именем, дописывает в него `listen 443 ssl` и пути к сертификату, а на 80 создаёт отдельный блок с перенаправлением. Всё, что вы настроили ниже — проксирование, пределы, сроки, — переезжает в HTTPS вместе с блоком. Готовый блок на 443 в образце был бы не заботой, а ошибкой: `listen ... ssl` без `ssl_certificate` nginx не принимает вовсе, и `nginx -t` не прошёл бы ещё до certbot.

Три вещи в этом файле не косметические, и менять их не стоит:

Строка	Почему
<code>proxy_set_header X-Forwarded-For \$proxy_add_x_forwarded_for</code>	Дописывает адрес клиента к цепочке. Без него (или с <code>\$remote_addr</code>) адресом клиента для системы станет прокси — один на всех, — и предел частоты входов схлопнется в общую корзину: пять неудачных попыток любого сотрудника закроют вход всей организации (\$10)
<code>client_max_body_size 600m</code>	Не меньше <code>BACKUP_MAX_ARCHIVE_MB</code> плюс запас. Прокси строже приложения делает его предел недостижимым, а отказ приходит HTML-страницей nginx без поля <code>error</code> — интерфейс покажет голое сообщение axios
<code>proxy_read_timeout в /api/backup/</code>	Восстановление из копии — минуты в одном запросе. На общих 300 секундах обрыв выглядит как неудача, хотя восстановление идёт и доходит до конца

В.3. Сертификат

```
sudo apt install -y certbot python3-certbot-nginx
sudo certbot --nginx -d docs.example.ru      # спросит почту для уведомлений об истечении
```

Certbot сам допишет в конфигурацию строки сертификата и перенаправление с HTTP. **Вписывать `ssl_certificate` заранее вручную не нужно** — конфигурацию, собранную не им, он не узнает и обновлять не станет.

Проверка, что продление будет работать (сертификат живёт 90 дней, продлевается за 30 до конца):

```
sudo certbot renew --dry-run
systemctl list-timers | grep certbot           # таймер ставится установкой пакета
```

В.4. Закрывать прямой вход

Пока порт `WEB_PORT` отвечает снаружи, рядом с HTTPS открыт вход по HTTP — весь контур в обход. Закрывается **переменной**, а не правкой compose-файла: файл приезжает ассетом релиза и при обновлении перезаписывается, унося правку с собой — молча.

```
# .env
WEB_BIND=127.0.0.1
```

```
docker compose up -d web
curl -s http://<внешний-адрес>:8080/api/version  # должно НЕ отвечать
curl -s https://docs.example.ru/api/version    # должно ответить версией
```

В.5. Сказать системе, что она за прокси

Две переменные в `.env`, обе — про то, что адрес системы изменился:

```
APP_PUBLIC_URL=https://docs.example.ru
```

- **APP_PUBLIC_URL** — из него собираются ссылки в письмах (сброс пароля, подтверждение адреса). Останется прежним — письма будут звать на `http://...` и старый порт, то есть в обход прокси. Не задан вовсе — письмо со сбросом пароля **не отправляется** (§8.4).
- **FORWARDED_TRUSTED_NETWORKS** — трогать не нужно, и это важнее, чем кажется. Переменная говорит, чьему `X-Forwarded-For` верить, а запрос приходит к `api` **не от вашего прокси напрямую**: между ними стоит контейнер `web`, и адресом отправителя для `api` всегда будет его адрес в сети Compose (172.16/12). Значение по умолчанию (пусто) уже покрывает петлю и частные диапазоны, то есть работает. А «сузить до прокси», вписав `127.0.0.1/32`, значит не оставить в списке ни одного адреса, который `api` вообще видит: заголовок будет отброшен, и предел частоты входов схлопнется в общую корзину — та самая беда, от которой этот заголовок и передаётся. Если сужение всё же нужно, берите подсеть сети Compose: `docker network inspect bhs-crg_default -f '{{(index .IPAM.Config 0).Subnet}}'`.

```
docker compose up -d api
```

После этого зайдите в систему по `https://` и проверьте: замок в адресной строке, «Скопировать» в любом списке работает, в форме «Сообщить об ошибке» появилась кнопка «Снять экран».

В.6. Сеть без выхода в интернет

Let's Encrypt там неприменим: проверить владение доменом ему нечем. Защищённый контекст при этом всё равно нужен — иначе часть интерфейса останется нерабочей. Два пути:

1. **Внутренний УЦ.** Если в организации он есть, выпустите сертификат на внутреннее имя и раздайте корневой сертификат на рабочие места групповой политикой. Для пользователя всё выглядит как обычный HTTPS.
2. **Самоподписанный сертификат.** Годится для стенда и небольшой установки; корневой сертификат придётся один раз добавить в доверенные на каждой машине, иначе браузер будет предупреждать при каждом входе, а часть возможностей всё равно останется отключённой.

```
sudo openssl req -x509 -nodes -days 825 -newkey rsa:2048 \  
-keyout /etc/ssl/private/bhs-crg.key -out /etc/ssl/certs/bhs-crg.crt \  
-subj "/CN=docs.internal" -addext "subjectAltName=DNS:docs.internal"
```

Certbot здесь не участвует, поэтому блок на 443 составляете вы сами: возьмите из образца тот, что на 80, поменяйте в нём первые строки и добавьте пути к сертификату — остальное (проксирование, пределы, сроки) остаётся как есть.

```
server {  
    listen 443 ssl;  
    listen [::]:443 ssl;  
    server_name docs.internal;  
  
    ssl_certificate      /etc/ssl/certs/bhs-crg.crt;  
    ssl_certificate_key  /etc/ssl/private/bhs-crg.key;  
  
    # ... дальше содержимое блока из образца: client_max_body_size, location / и location  
    /api/backup/  
}  
  
# И перенаправление с HTTP — то, что при Let's Encrypt делает certbot  
server {  
    listen 80;  
    listen [::]:80;  
    server_name docs.internal;  
    return 301 https://$host$request_uri;  
}
```

http2 on; добавляйте, только если у вас nginx **1.25.1 и новее**; на более старых (Ubuntu 22.04 везёт 1.18, Debian 12 — 1.22, Ubuntu 24.04 — 1.24) эта запись останавливает запуск с «unknown directive», а включается HTTP/2 там иначе: `listen 443 ssl http2;`. Проверено на всех трёх.

О «Всё равно перейти». Возможности интерфейса после такого перехода работают: браузер считает контекст защищённым по схеме `https://`, и буфер обмена, снимок экрана и `randomUUID` доступны (проверено в Chromium: недоверенный сертификат, переход вопреки предупреждению — `isSecureContext: true`). Так что «работает же» здесь не аргумент ни за, ни против.

Против — другое. Предупреждение встречает каждого пользователя при каждом входе, и человек, приученный жать «Всё равно перейти», не отличит настоящую подмену от привычного неудобства: ровно то, от чего защищает HTTPS, перестаёт работать — не технически, а в голове. Поэтому

корневой сертификат добавляют в доверенные **на машинах пользователей**, а не учат обходить предупреждение.

В.7. Если что-то не так

Симптом	Причина
<code>nginx -t: no "ssl_certificate" is defined for the "listen ... ssl"</code>	В конфигурации появился блок на 443 без путей к сертификату. При Let's Encrypt их дописывает certbot — до него блока на 443 быть не должно (В.2); при своём сертификате пути вписываются вручную (В.6)
<code>nginx -t: unknown directive "http2"</code>	Запись <code>http2 on;</code> понимает nginx 1.25.1 и новее; на 1.18/1.22/1.24 — <code>listen 443 ssl http2;</code>
Certbot: «Timeout during connect»	Порт 80 закрыт снаружи или А-запись ведёт не сюда. Проверить: <code>curl -I http://docs.example.ru</code> с другой машины
Открывается стандартная страница nginx	Не удалён <code>/etc/nginx/sites-enabled/default</code> — он перехватывает имя первым
В интерфейсе 502 Bad Gateway	Стек не отвечает на <code>WEB_PORT</code> . Проверить: <code>docker compose ps</code> и <code>curl -s http://127.0.0.1:8080/api/version</code> с сервера
«Слишком много попыток входа» у единичных пользователей	<code>X-Forwarded-For</code> не доходит: проверьте эту строку в конфигурации и <code>FORWARDED_TRUSTED_NETWORKS</code> (§10)
Не загружается крупный файл	<code>client_max_body_size</code> на прокси ниже <code>BACKUP_MAX_ARCHIVE_MB</code>
Письма приходят со ссылками на <code>http://...</code> и старый порт	Не обновлён <code>APP_PUBLIC_URL</code> (В.5)